

Homework Assignment- Advanced Prompt Injection Attack and Defense Benchmark

This assignment is fully standalone. Students must design, implement, attack, defend, and evaluate a small LLM-enabled assistant without relying on instructor-provided code or datasets. The objective is not merely to demonstrate a prompt injection, but to construct a repeatable benchmark, measure security and usability, and critically analyse what makes a defense succeed or fail.

Learning Objectives

- Model a realistic prompt injection threat scenario for an LLM application.
- Design a benchmark containing both benign and adversarial prompts with clear labels and success criteria.
- Implement layered defenses rather than relying on a single keyword filter.
- Measure trade-offs among attack success rate, false refusals, usefulness, and latency.
- Perform failure analysis and propose grounded improvements.

Provided Material

- Students must create their own application, prompt corpus, and evaluation logs.
- Open-source models, hosted APIs, public prompt collections, and publicly documented defense ideas may be used, but every external dependency must be cited and justified.

Required Tasks

1. Build a Baseline Assistant (Core Task)

Create a small assistant that performs at least two benign user tasks while also containing protected information that should never be revealed to the user.

Steps:

- Implement an assistant with a hidden system instruction or hidden policy text.
- Store at least three protected items server-side or in a hidden context. These may be dummy flags, confidential notes, or protected policy strings created by the student.
- Support at least two legitimate user-facing tasks such as summarisation, question answering, formatting, or classification so the system has a non-trivial benign purpose.
- Log every request and response with timestamps and a unique interaction identifier.
- Write a short threat model describing the attacker goal, what counts as a successful compromise, and what benign functionality must remain usable.

Minimum evidence expected:

- A runnable baseline system with documented setup instructions.
- A short architecture description showing where user input, system instructions, hidden data, and logs are stored.

2. Construct an Evaluation Benchmark

Design a benchmark that allows the security of the application to be measured rather than judged informally.

Steps:

- Create at least 60 test cases: a minimum of 30 benign prompts and 30 adversarial prompts.
- Use at least five adversarial families, such as direct instruction override, role-play, indirect or quoted instruction injection, multilingual or translated attacks, encoded or obfuscated prompts, and multi-turn decomposition.
- For each test case, store a label, attack family, intended objective, and expected safe behaviour.
- Include at least five multi-turn attack sequences where the compromise only becomes possible after context has been established across several turns.
- Define an explicit success criterion for each adversarial test, such as secret leakage, policy violation, unsafe tool invocation, or refusal bypass.

Minimum evidence expected:

- A benchmark file in CSV or JSON format that can be reused by another student or marker.
- A short explanation of how difficulty levels or categories were assigned.

3. Benchmark the Baseline System

Run the full benchmark against the unprotected or minimally protected assistant and produce quantitative results.

Steps:

- Evaluate the entire benchmark automatically or semi-automatically and save the outputs.
- Report attack success rate overall and per attack family.
- Report benign task success rate and false refusal rate for the benign prompts.
- Measure average latency or throughput for benign and adversarial requests.
- Select at least ten representative interactions, including successful attacks, blocked attacks, benign successes, and benign failures, and discuss what happened.

Minimum evidence expected:

- A baseline results file with enough evidence for the metrics stated in the report.
- At least one concise visual summary such as a table or chart comparing attack families.

4. Design and Implement Layered Defenses

Improve the assistant by combining multiple defenses and by clearly separating defensive goals from application functionality.

Steps:

- Implement at least three distinct defense layers.
- At least one defense must operate before generation, for example input normalisation, canonicalisation, suspicious-pattern detection, or structured prompt packaging.
- At least one defense must improve context separation, for example hidden state isolation, trusted versus untrusted channel separation, or constrained tool invocation.
- At least one defense must operate after generation, for example output filtering, a policy checker, or a secondary judge model.
- Justify why each defense targets a different failure mode and explain its likely limitations.
- Document any thresholds, heuristics, or model choices used in the defense stack.

Minimum evidence expected:

- An updated application or pipeline that clearly distinguishes baseline and defended behaviour.
- A brief design note explaining how the defenses interact and where they can still fail.

5. Comparative Evaluation and Ablation Study

Re-run the benchmark and determine whether the defenses actually help, and which defense contributes the most.

Steps:

- Evaluate the defended system on the same benchmark used for the baseline.
- Compare baseline and defended results using the same metrics and the same definitions.
- Perform an ablation study by removing one defense layer at a time, or by comparing at least three configurations: baseline, partially defended, and fully defended.
- Identify any cases where security improved but usability worsened, for example an increase in benign refusals.
- Discuss whether the defense stack generalises across attack families or mainly blocks only the attacks it was tuned for.

Minimum evidence expected:

- A comparative results table covering all tested configurations.
- A discussion of trade-offs rather than a single final score.

6. Critical Error Analysis

Perform a detailed analysis of remaining failures rather than simply reporting final metrics.

Steps:

- Include at least two false positives, two false negatives, and two partial failures or ambiguous cases.
- Link each failure to a plausible root cause such as weak threat modeling, overfitting to known attacks, insufficient context isolation, model behaviour, or brittle rules.
- Explain whether the failure is likely model-specific or architecture-specific.
- Propose concrete next-step improvements and discuss how they would be validated experimentally.
- State clearly what security claims you can support with evidence and what claims remain unproven.

Minimum evidence expected:

- A structured error-analysis section in the report, not just a list of mistakes.

7. User-Facing Improvement or Interpretable Output Extension

Add at least one feature that makes the defended system easier to inspect, explain, or use safely.

Steps:

- Add one visible feature such as a risk score, blocked-pattern explanation, defense trace, prompt classification label, or evaluation dashboard.
- Ensure the new feature is interpretable and does not reveal the protected secrets themselves.
- Demonstrate the feature on at least three benchmark examples and explain why it improves transparency or usability.

Minimum evidence expected:

- Screenshots or console captures showing the extension in action.

Deliverables

- Updated source code for the application, benchmark runner, defenses, and logging pipeline.
- The benchmark file containing the benign and adversarial prompts.

- Results files or logs sufficient to reproduce the reported metrics.
- A structured report covering system design, threat model, benchmark construction, evaluation, ablation, and error analysis.
- Screenshots of the baseline and defended system outputs.
- A requirements file and a README that explains exactly how the marker can run the project.

Submission Guidelines

- The submission must run without undocumented manual steps.
- If external APIs are used, state the exact model, provider, date of access, and how the API dependency affects reproducibility.
- Students may include small example outputs, but should not submit unnecessary large model weights.
- The report should distinguish clearly between measured results and assumptions or hypotheses.

Optional Advanced Tasks

- Implement an adaptive attacker that automatically rewrites failed prompts and measures whether the defense continues to hold.
- Compare at least two different base models or two deployment settings, for example local open-source inference versus hosted API inference.
- Create a multilingual benchmark and analyse whether defenses transfer across languages.
- Add a judge model or secondary verifier and evaluate whether it improves precision without sharply increasing benign refusals.
- Extend the application with tool use or retrieval, then evaluate how the additional capability changes the attack surface.